CRIMEVISIONAI

Data Flow Diagram

Folder 2 | Logical data flow for the current prototype | Prepared 23 October 2026

Project: CrimeVisionAI — Multimodal Forensic Evidence Analysis and Investigation Support System

Document status: Project-specific draft based on the implementation information currently available. Verify placeholders and run-dependent results before formal submission.

1. Context and flow

CrimeVisionAI receives user-selected evidence through the browser UI, processes it through a Flask backend, stores case/evidence metadata in SQLite and files on local disk, invokes modality-specific models and the Groq LLM integration, then presents analysis outputs to the reviewer.

2. Level-0 context diagram

External entity / store	Data entering / leaving	Interaction
Investigator (browser)	Case requests, evidence uploads, analysis requests, questions; receives results	HTML/CSS/JavaScript UI communicates with Flask routes.
CrimeVisionAI Flask application	Validated requests and analysis orchestration	Routes connect the UI, database, filesystem and model functions.
Local file store	Uploaded audio/image/video; media retrieval	Evidence files stored locally, including case-scoped uploads.
SQLite database	Cases, evidence metadata, analysis results	Tables reported: cases, evidence, analysis_results.
Local AI models	Audio, image and sampled video-frame inputs; transcripts, detections, captions	Whisper, RT-DETR, CLIP and BLIP pipelines.
Groq API	Prompt/context derived from analysis; structured interpretation / Q&A response	Model ID reported as openai/gpt-oss-20b; requires configured API access.

3. Level-1 processing sequence

Step	Process	Output / persistence
1	Browser sends case/evidence request to Flask.	Request validated; errors returned for invalid input.
2	Upload handler validates supported type/size and saves file.	Local evidence file and evidence metadata.
3	Analysis route selects audio, image or video pipeline.	Transcript / timestamps, object detections, captions, scene labels and other model output.
4	LLM integration structures event candidates or answers questions using available analysis context.	Generated interpretation / candidate events / answer; must be reviewed for grounding.
5	Analysis result is associated with evidence and case records.	SQLite analysis_results and linked evidence/case records, according to current schema.
6	UI retrieves evidence, media and analysis outputs.	Reviewer inspects results and timeline; Q&A grounding remains a known quality gap.

4. Data handling and controls

Current implementation details reported: 100 MB Flask request limit; audio processed in 30-second chunks; video sampled at 1 frame per second and limited to 180 seconds / 180 frames. These are application limits, not performance guarantees. Confirm that file-type checks, path handling, error cleanup and secret management are enforced in the current code before production use.

5. Important limitation

The diagram describes the known logical architecture; it is not a claim that the application is deployed as separate microservices. The current baseline is a Flask application with local models, SQLite and local filesystem storage.